EKS Implementation Patterns and Controls

Version : 0.01 First Draft
Date : 2024-04-30
Author : Bobby.Willoughby@transamerica.com

Version control

Version	Date	Author/updated by	Comments
0.01	2024-04-30	Bobby Willoughby	First Draft

Note:

Dates in ISO format: YYYY-MM-DD

Versions: 0.XX Pre-release, n.XX Release, n.XX.xx draft releases of version under version control

Contents

Contents **3**

Overview **5**
Drivers and Motivations 5
Business Value 5

EKS Model Selection Considerations **7**
Risks and Security Considerations and Characteristics of Implementation Models 7
Risks to be considered with an EKS deployment 7
Characteristics of the four EKS models in use at Aegon 10
Other Scenarios to Consider 11
Cost Optimization 12
Operational Excellence 13
Reliability 13
Performance Efficiency 13
Sustainability 14

Functionality / How it Works **15**
Anatomy of EKS 15
Deployment Models and Security Controls 16
Security Controls in EKS 16
Non-Routable Nodes and Containers Isolated to the VPC 17
Non-Routable Nodes and Containers with controlled access outside the VPC 18
Routable Nodes with non-routable containers 19
Routable Nodes and Containers 20

Architectural Attributes **21**
Interfaces 21
Redundancy, Availability & Reliability 21
Scale, Elasticity & Capacity 22
Cost, Business & Support Models 23

User Access & Roles **24**
Federated Roles 24
Access Control Policies 24
Restrictions 24
Permitted Regions 24

Use Cases **25**

Data & Governance **26**
Data Classification 26
Guardrails 27
No Public Access 27

Container NAT to the Node Primary IP Address is Disabled 27

Observability 28

How Observability Can Be Performed 28

How Observability Tools Work Together 29

Product / Component Selection 30

Feature Comparison 30

Exceptions 30

Usage in Aegon 31

Related Component Standards 31

Architectural Patterns 31

Reference Architecture 31

Solution Architectures 31

Overview

This standard will define the multiple implementation patterns for AWS EKS (Elastic Kubernetes Service). EKS is a container service in AWS where multiple containers can run on EC2 servers. Each implementation model will have security controls documented to properly secure the environment. This component standard is a building block for solution architectures requiring Kubernetes on AWS.

AWS EKS (Elastic Kubernetes Service) is a managed service provided by Amazon Web Services (AWS) that simplifies the deployment, management, and scaling of containerized applications using Kubernetes. It allows users to run Kubernetes on AWS without the need to install, operate, and maintain the Kubernetes control plane. This standard is specific to AWS EKS but much of it may apply to other Kubernetes management platforms.

AWS EKS can be defined as a cloud-based container orchestration service that provides a highly available, scalable, and secure platform for running containerized applications. It abstracts the underlying infrastructure and automates many tasks related to managing Kubernetes clusters, such as provisioning, scaling, and monitoring. With AWS EKS, developers can focus on building and deploying applications, while AWS takes care of the underlying Kubernetes infrastructure.

The implementation of EKS can be accomplished using multiple potential models. Each model has a varying degree of security and operational overhead. This standard will discuss the following implementation models; ordered from most to least secure.

1. *Non-Routable Nodes and Containers Isolated to the VPC*
2. *Non-Routable Nodes and Containers with controlled access outside the VPC*
3. *Routable Nodes with non-routable containers*
4. *Routable Nodes and Containers*

The security model for EKS varies slightly depending on the implementation model chosen. Other than isolation of the resources, there are only two primary ways to secure the flow of traffic; Security Groups and EKS Network Policies.

Drivers and Motivations

EKS is a very desirable container service provided by AWS and offers several different implementation patterns that need to be carefully considered as the business looks to implement the service. It is important that we consider how the different resources in the EKS architecture interoperate and what controls are possible for protecting/segmenting the applications running in EKS. These will be discussed in this section: Risks, Security Considerations

The business drivers for AWS EKS as a container service include simplifying deployment, scalability, high availability, security, automation, and cost-effectiveness.

Business Value

The business values gained by using AWS EKS as a container service include:

- **Simplified Deployment**: AWS EKS simplifies the deployment process of containerized applications using Kubernetes. It eliminates the need for users to manually install, operate, and maintain the Kubernetes control plane, reducing complexity and saving time. For further abstraction and simplification AWS Fargate might be considered.

- **Scalability**: EKS provides a highly scalable platform for running containerized applications. It allows users to easily scale their applications up or down based on demand, ensuring optimal performance and resource utilization.
- **High Availability**: EKS offers a highly available platform for running containerized applications. It automatically manages the underlying infrastructure and provides redundancy and fault tolerance, minimizing downtime and ensuring continuous availability.
- **Security**: EKS provides a secure platform for running containerized applications. It abstracts the underlying infrastructure and automates security-related tasks, such as patching and updates, ensuring that applications are running in a secure environment.
- **Automation**: EKS automates many tasks related to managing Kubernetes clusters, such as provisioning, scaling, and monitoring. This automation reduces the operational burden on developers and allows them to focus on building and deploying applications.
- **Cost-Effectiveness**: EKS helps optimize costs by providing a managed service for running containerized applications. It eliminates the need for users to invest in and maintain their own Kubernetes infrastructure, reducing capital and operational expenses.

EKS Model Selection Considerations

EKS is one of a handful of container management solutions for consideration. Each of the cloud providers have a similar implementation that supports Kubernetes. Many of the benefits of EKS are generated just by running in the AWS environment. Like most resources in AWS, there is a shared responsibility model for operations and security. For EKS, the operations responsibility shifts further toward AWS as the business considers Fargate.

Risks, Security Considerations and Characteristics of Implementation Models

EKS Security is provided with several controls and varies depending on the implementation model chosen. The two most effective controls for network traffic are Security Groups and EKS Network Policies. You can find more information about these controls here: Deployment Models and Security Controls.

Risks to be considered with an EKS deployment

EKS Cluster/Controller Risks

Risks associated with the potential compromise of the EKS Controller/Cluster within the VPC and as part of the Aegon network:

- **Unauthorized access**: A compromised EKS controller can allow attackers to gain unauthorized access to the EKS cluster and potentially compromise other resources within the cluster.
- **Data breaches**: Attackers can exploit a compromised EKS controller to access and steal sensitive data stored within the cluster, leading to potential data breaches.
- **Control over the cluster**: A compromised EKS controller can give attackers control over the entire cluster, allowing them to manipulate or disrupt the operation of applications running on the cluster.
- **Malicious activities**: Attackers can use a compromised EKS controller to launch further attacks within the cluster, such as deploying malicious containers or executing unauthorized commands.
- **Loss of availability**: A compromised EKS controller can result in the loss of availability of the cluster, impacting the availability of applications and services running on the cluster.

To mitigate these risks, it is important to implement strong security measures such as:

- Regularly updating and patching the EKS controller to ensure it is running the latest secure version.
- Implementing strong access controls and authentication mechanisms to restrict access to the EKS controller.
- Monitoring the EKS controller for any suspicious activities or anomalies.
- Implementing network segmentation and isolation to limit the impact of a compromised EKS controller.
- Implementing strong encryption mechanisms to protect data in transit and at rest within the EKS cluster.
- Implementing robust logging and monitoring solutions to detect and respond to any security incidents or compromises.

EKS AMI Risks

Risks associated with the AMI (Amazon Machine Image) used to inflate the EKS node:

- **Vulnerabilities**: The AMI may contain outdated software or configurations that have known security vulnerabilities, which can be exploited by attackers.
- **Malware or Backdoors**: The AMI could be compromised and contain malware or backdoors that can be used to gain unauthorized access or control over the EKS node.
- **Misconfigurations**: The AMI may have misconfigurations that can lead to security weaknesses, such as open ports or weak authentication mechanisms.
- **Lack of Patching**: If the AMI is not regularly updated and patched, it may be susceptible to known security issues that have been fixed in newer versions.

To mitigate these risks, it is important to follow security best practices when selecting and using AMIs for EKS nodes. This includes:

- Using official and trusted AMIs provided by GTS.
- Regularly updating and patching the AMIs to ensure they have the latest security fixes.
- Regularly scan the AMIs for vulnerabilities.
- Implementing strong security controls, such as proper network segmentation, access control, and encryption, to protect the EKS nodes and the data they handle.

EKS Container Risks

There are quantifiable risks that we should consider specific to the EKS container.

- **Container vulnerabilities**: Vulnerabilities can be exploited by attackers and lead to unauthorized access, data breaches, or disruption of services.
- **Container escape**: In certain scenarios, attackers may be able to escape the container and gain access to the underlying host or other containers running on the same host. This can result in unauthorized access or compromise of other containers.
- **Container misconfigurations**: Misconfigurations in container settings or permissions can lead to security weaknesses, such as excessive privileges or insecure network configurations. Misconfigurations may be used to gain unauthorized access or perform malicious activities.
- **Container image security**: The security of the container images used in EKS is crucial. If the images are compromised or contain malware, it can lead to unauthorized access, data breaches, or disruption of services.

To mitigate these risks, it is important to implement the following measures:

- Regularly update and patch container images to ensure they are running the latest supported versions.
- Implement strong access controls and authentication mechanisms for containers.
- Use container security tools and practices, such as vulnerability scanning, image signing, and runtime monitoring.
- Implement network segmentation and isolation to limit the impact of a compromised container.
- Follow security best practices for container deployment and configuration.
    - Use trusted container images: Only use container images from trusted sources and regularly update and patch them to ensure they are running the latest secure versions.
    - Implement strong access controls: Use strong authentication mechanisms and access controls to restrict access to containers. This includes using strong passwords, multi-factor authentication, and role-based access control (RBAC).
    - Limit container privileges: Configure containers with the least privileges necessary to perform their intended functions. Avoid running containers with root privileges whenever possible.
    - Secure container networking: Implement secure network configurations for containers, such as using encrypted communication channels, isolating containers from each other, and using network segmentation to limit communication between containers.
    - Regularly monitor and scan for vulnerabilities: Use container security tools and practices, such as vulnerability scanning and monitoring, to identify and address any vulnerabilities or misconfigurations in containers.
    - Implement container runtime security: Use container runtime security tools and practices, such as container isolation technologies (e.g., Kubernetes namespaces) and runtime protection mechanisms, to prevent container escapes and unauthorized access to the underlying host.
    - Implement logging and monitoring: Enable logging and monitoring for containers to detect and respond to any suspicious activities or anomalies. This includes monitoring container logs, network traffic, and system metrics.
    - Regularly backup container data: Implement regular backups of container data to ensure that in the event of a compromise or data loss, you can restore the containers to a known secure state.

Characteristics of the four EKS models in use at Aegon

This document provides a description of four basic EKS implementation models:

Non-Routable Nodes and Containers Isolated to the VPC

The characteristics of this model are:

- the nodes (EC2) and containers are both running in VPC Subnets that are using a non-routable IP Address space (100.64.0.0/16).
- the Subnets are not using NAT Gateways to reach outside the VPC to other resources.
- the Subnets are not using Load Balancers to allow resources outside the VPC to reach into the EKS nodes/containers.

This is the most secure option as the EKS containers and nodes are restricted to communicating with the resource in the same VPC only. In this model, additional security should be considered to isolate the EKS resources from the rest of the data/compute resources using the least-privilege security model. Even with this model, additional security controls need to be considered when multiple Applications, Namespaces and/or Clusters are used in the same VPC.

Non-Routable Nodes and Containers with controlled access outside the VPC

The characteristics of this model are:

- the nodes (EC2) and containers are both running in VPC Subnets that are using a non-routable IP Address space (100.64.0.0/16).
- the Subnets are using a NAT Gateways to reach outside the VPC to other resources.
- and/or the Subnets are using Load Balancers to allow resources outside the VPC to reach into applications hosted by the EKS cluster.

This model is slightly less secure than the previous option as the EKS cluster is reachable on the Aegon network through the NAT Gateways and Load Balancers. In this model, additional security should be considered to isolate the EKS resources from the rest of the data/compute resources using the least-privilege security model. Even with this model, additional security controls need to be considered when multiple Applications, Namespaces and/or Clusters are used in the same VPC. As the NAT Gateways and Load Balancers can communicate outside the VPC Network, Segmentation Policies should be applied to limit access to/from the EKS Nodes and applications.

Routable Nodes with non-routable containers

The characteristics of this model are:

- the nodes (EC2) are running in VPC Subnets that are using a routable IP Address space.
- the containers are running in VPC Subnets that are using a non-routable IP Address space (100.64.0.0/16). This implies that the node has a secondary ENI in the non-routable subnet.
- the Subnets may be using a NAT Gateways to reach outside the VPC to other resources.
- and/or the Subnets may be using Load Balancers to allow resources outside the VPC to reach into applications hosted by the EKS cluster.
- the default behavior for the containers to NAT traffic through the primary interface of the node is disabled. If this behavior is NOT disabled, and exception is required, and the environment needs to be treated like Routable Nodes and Containers.

This model is slightly less secure than the previous options as the EKS node is reachable on the Aegon network. As with the previous option, the EKS cluster may be reachable on the Aegon network through the NAT Gateways and Load Balancers. In this model, additional security should be considered to isolate the EKS resources from the rest of the data/compute

resources using the least-privilege security model. Even with this model, additional security controls need to be considered when multiple Applications, Namespaces and/or Clusters are used in the same VPC. As the Nodes, NAT Gateways and Load Balancers can communicate outside the VPC Network, Segmentation Policies should be applied to limit access to/from the EKS Nodes and applications.

Routable Nodes and Containers

The characteristics of this model are:

- the nodes (EC2) and containers are running in VPC Subnets that are using a routable IP Address space.
- this model has the highest demand on routable IP Address space in a VPC and will be limited to 4k hosts (/21 CIDR on VPC).
- an exception is required to use this pattern.

This is the least intrinsically secure model as all the EKS nodes and containers are reachable on the Aegon network. In this model, security should be considered to isolate the EKS resources from the rest of the data/compute resources using the least-privilege security model. Within this model, additional security controls need to be considered when multiple Applications, Namespaces and/or Clusters are used in the same VPC. As all the EKS resources can communicate outside the VPC, Network Segmentation Policies should be applied to limit access to/from the VPC. Network restrictions are required at the VPC level because the EKS containers fluctuate up/down so frequently automation cannot keep up with the resources change rate in the firewall environment.

Other Scenarios to Consider

Multiple Applications in the same Cluster

Security between applications/containers associated in the same cluster can only be provided when the applications/containers are associated with different Namespaces. While AWS best practices indicates that a single EKS Cluster should be used to support a single application and that each EKS Cluster is in a separate VPC, if the business chooses to implement EKS in a way contrary to this advice security will need to be implemented using Namespaces. See the next section for more on implementing security then there are **Error! Not a valid bookmark self-reference.**.

Multiple Namespaces in a Cluster

Security between applications/containers associated with different Namespaces can be implemented within the EKS environment using a Container Network Interface (CNI). A CNI is a virtual interface for the container associated with a secondary Elastic Network Interface (ENI) on the EC2 Server/EKS node. Traffic between containers of different Namespaces can be filtered with an EKS Network Policy at the CNI. Multiple applications in a single cluster should be associated with different Namespaces.

Multiples Clusters in same VPC

Security between applications/containers associated with different Clusters can only be achieved at Aegon using VPC Security Groups and Network ACLs. While it is possible to implement clusters in different subnets and route traffic between those subnets through a firewall, there is not an Aegon security pattern supporting intra-VPC firewalls control.

To implement segmentation between clusters, the Security Groups associated with the primary ENI of each EKS node will need to be configured using a least-privilege model.

Cost Optimization

One of the greatest benefits of containers in general and EKS specifically, is the cost-effectiveness or the solution. As stated in the values section, EKS helps optimize costs by providing a managed service for running containerized applications. It eliminates the need for users to invest in and maintain their own Kubernetes infrastructure, reducing capital and operational expenses. Further cost optimization may be generated by considering Fargate.

EKS (Amazon Elastic Kubernetes Service) provides several features and capabilities that can help with cost optimization:

- **Auto Scaling**: EKS allows you to configure auto scaling for your worker nodes. This means that the number of worker nodes can automatically scale up or down based on the demand for your applications. By scaling down during periods of low demand, you can save on costs.
- **Spot Instances**: EKS supports the use of EC2 Spot Instances for worker nodes. Spot Instances are spare EC2 instances that are available at a significantly lower cost compared to On-Demand instances. By using Spot Instances, you can reduce your compute costs. However, it's important to note that Spot Instances can be interrupted with a short notice, so they are best suited for fault-tolerant and stateless workloads.
- **Reserved Instances**: If you have predictable and steady-state workloads, you can purchase Reserved Instances for your EKS worker nodes. Reserved Instances provide a significant discount compared to On-Demand instances, allowing you to save costs over a longer period.
- **Cluster Autoscaler**: EKS integrates with the Kubernetes Cluster Autoscaler, which automatically adjusts the size of your worker node group based as the resource demands of your application changes. This helps to optimize costs by ensuring that you have the right number of resources allocated to your applications at any given time.
- **Cost Allocation Tags**: EKS supports cost allocation tags, which allow you to categorize and track your costs based on different dimensions such as teams, projects, or environments. By using cost allocation tags, you can gain better visibility into your EKS costs and optimize spending based on specific areas of your organization.
- **Monitoring and Optimization Tools**: EKS integrates with various monitoring and optimization tools such as Amazon CloudWatch, AWS Cost Explorer, and AWS Trusted Advisor. These tools provide insights into your EKS costs, identify potential cost-saving opportunities, and help you optimize your resource allocation.

Operational Excellence

EKS (Amazon Elastic Kubernetes Service) is operationally excellent due to several factors:

- **Managed Service**: EKS is a fully managed service provided by Amazon Web Services (AWS). This means that AWS takes care of the operational aspects of managing the Kubernetes control plane, including upgrades, patches, and scaling. This allows users to focus on their applications rather than the underlying infrastructure.
- **High Availability**: EKS ensures high availability by distributing the Kubernetes control plane across multiple availability zones (AZs). This provides fault tolerance and ensures that the control plane remains operational even if an AZ goes down.
- **Scalability**: EKS allows for easy scaling of worker nodes to meet the demands of applications. With auto scaling capabilities, the number of worker nodes can automatically scale up or down based on the workload, ensuring optimal resource utilization and cost efficiency.
- **Integration with AWS Services**: EKS seamlessly integrates with other AWS services, such as Elastic Load Balancing, Amazon RDS, Amazon S3, and more. This allows users to leverage the full range of AWS services to enhance their applications and simplify operations.
- **Monitoring and Logging**: EKS provides integration with AWS CloudWatch for monitoring and logging. Users can easily monitor the performance and health of their clusters, set up alarms, and analyze logs to troubleshoot issues and optimize their applications.
- **Security**: EKS offers robust security features, including IAM roles and policies for access control, VPC networking for isolation, and integration with AWS Identity and Access Management (IAM) for authentication and authorization. Users can implement fine-grained security controls to protect their applications and data using the Container Network Interface and EKS Network Policies.

Reliability

Aegon should expect the EKS Environment to be highly reliable. AWS provides for high reliability based on similar capabilities as defined in Operational Excellence. High Availability and Monitoring are most important to the reliability of the infrastructure supporting the EKS environment.

Performance Efficiency

Aegon should expect the EKS Environment to be highly performant. When using the scaling capabilities of EKS, the environment can scale to meet the demands of the worker nodes. One note though, if Spot Instances are being used, scaling may be impacted in times of high demand, as the infrastructure may not scale as fast or as large as needed for the given workload. AWS provides for high reliability based on similar capabilities as defined in Operational Excellence.

Sustainability

From an environmental perspective, the EKS environment is sustainable due to the following factors:

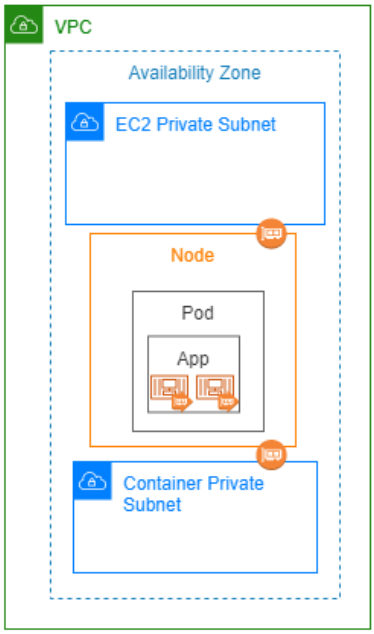
- **Resource Optimization**: EKS allows for easy scaling of worker nodes based on application demands. This ensures optimal resource utilization, minimizing waste and reducing the overall environmental impact.
- **Efficient Infrastructure Management**: As a fully managed service, AWS takes care of the operational aspects of managing the Kubernetes control plane. This includes upgrades, patches, and scaling, which are done in an efficient manner, reducing energy consumption and carbon footprint.
- **Server Consolidation**: By distributing the Kubernetes control plane across multiple availability zones, EKS ensures high availability and fault tolerance. This allows for server consolidation, reducing the number of physical servers required and optimizing energy usage.
- **Auto Scaling**: EKS's auto scaling capabilities enable the platform to dynamically adjust the number of worker nodes based on workload. This ensures that resources are allocated efficiently, preventing overprovisioning and unnecessary energy consumption.

Functionality / How it Works

Anatomy of EKS

AWS EKS is composed of several components that together make up a resource that can be virtualized to support multiple applications.

VPC Resources



Availability Zone: EKS can run across AZs, but each node is associated with a specific AZ.

EC2 Subnet: EKS can run across multiple subnets, but each node is associated with a specific subnet as the primary interface.

Container Subnet: EKS Containers may run in a subnet separate from the primary subnet. In this case the node has a second ENI configured and will be associated with a specific subnet in the same AZ as the primary interface.

EKS Resources

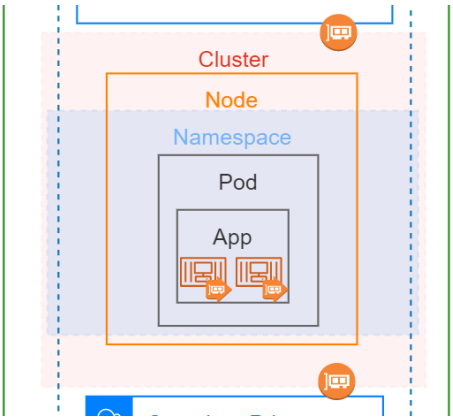
Node: A worker node is associated with a specific EC2 server. This is the largest compute resource in EKS. Since the node is associated with a EC2 server, it cannot span across AZs.

Pod: A Node can be split into multiple pods. As such, a pod is associated with a single node in a single subnet in a single AZ. A pod is the basic unit of deployment in EKS; the smallest and most basic deployable unit in Kubernetes. It is a group of one of more applications and represents the containers, storage and network configuration and can be scheduled to be deployed on multiple nodes.

App: An application is a grouping of one of more containers running in a pod within EKS. As a pod can be scheduled to run on multiple nodes, so can an app.

Container: The encapsulated instance of the application that runs within the worker node

Container Network Interface (CNI): A virtual network interface that gives the pod the ability to use a separate subnet for network communication from the primary node subnet. The CNI is required to enforce EKS Network Policies.



Cluster: A managed service provided by Amazon Web Services (AWS) that simplifies the deployment, management, and scaling of Kubernetes clusters. EKS clusters are used to run containerized applications using Kubernetes orchestration capabilities. They provide a scalable and highly available environment for running and managing containers.

Namespace: A virtual cluster that provides a way to divide and isolate resources within a cluster. They can be used to organize and manage resources, such as pods, services, and deployments, based on different criteria, such as environment, team, or application. Namespaces also help in resource quota management and access control.

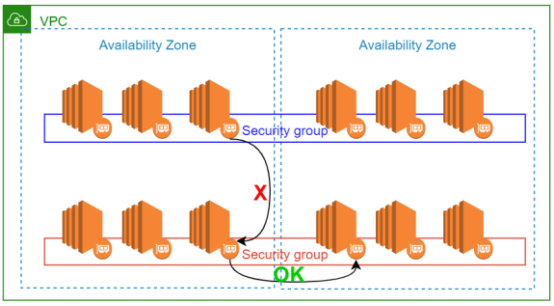
Deployment Models and Security Controls

At Aegon we currently support four different implementation models for EKS. Each model is based on the AWS Well Architected standards for EKS but have slight variations to meet differing business needs.

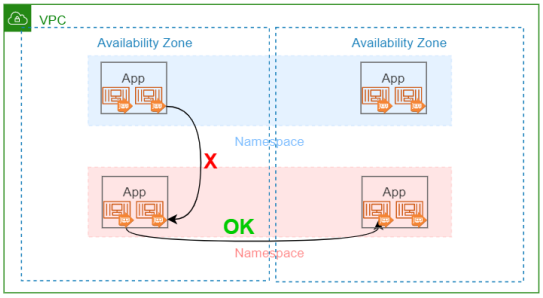
Security Controls in EKS

AWS EKS offers the ability to provide security controls within the VPC using the following methods recognized by Aegon:

Security Groups: SGs must be applied to the ENI (Network Interface) of every node. These SGs can be configured to only allow the node and containers to communicate to specific resources in the VPC. Is this example, objects in the same security group can communicate to with each other, but communication between the Blue and Red security groups can be filtered.



EKS Network Policies: With the use of a Container Network Interface (CNI) , a EKS Network Policy can be used to control access between different applications within a EKS Cluster. NameSpaces can also be used to group applications resources together that share a common security profile.



Non-Routable Nodes and Containers Isolated to the VPC

In this model, the EKS Worker Nodes and Containers are installed on private, non-routable subnets that are isolated to the VPC where the cluster is deployed. The nodes and containers can be using the same of different non-routable subnets. These resources cannot reach any resources outside the VPC, except for services available over VPC Endpoints.

While the EKS environment is isolated to the VPC, consideration should be given to the accessibility of other resources in the VPC to the EKS environment. This may not be a concern as any data access by the EKS is contained within the VPC, or resources directly associated with the VPC and AWS Account.

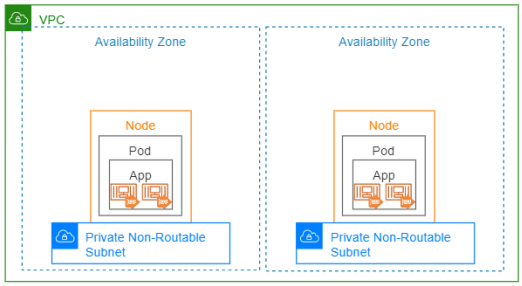


Figure 1: EKS deployed on Node with Default NATing enabled for Container

Figure 2: EKS deployed on Node with Default NATing enabled for Container (EC2 View)

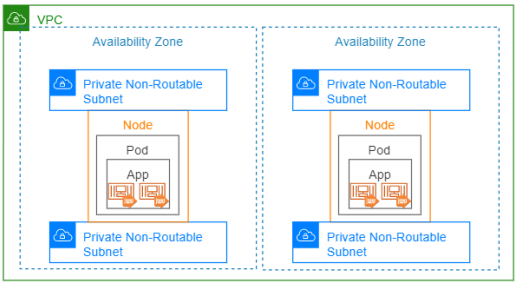


Figure 4: EKS deployed on Node with Dual Subnets

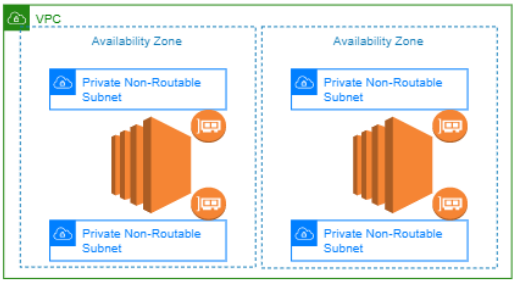


Figure 3: EKS deployed on Node with Dual Subnets (EC2 View)

Non-Routable Nodes and Containers with controlled access outside the VPC

In this model, the EKS Worker Nodes and Containers are installed on private, non-routable subnets as in the previous example. These resources are isolated to the VPC where the cluster is deployed. Other than AWS resources reachable over VPC Endpoints, these resources cannot reach any resources outside the VPC **directly**. This model does, however, offer the ability to allow communication to/from other Aegon networks with the use of additional network resources. In this model we deploy two separate network resources to facilitate this communication.

Egress: For access to resources outside the VPC, a NAT Gateway is deployed on a routable subnet in each availability zone. The route table of the EKS Subnets are modified to use the NAT Gateway as the route to all resources outside the VPC. With the routes in place, all traffic destined to the Aegon network (or Internet) is NATed to the NAT Gateway IP Address. When all these resources are in place the EKS resources can route over the Aegon network where segmentation policies can be applied to the NAT Gateways to inspect/restrict access as necessary.

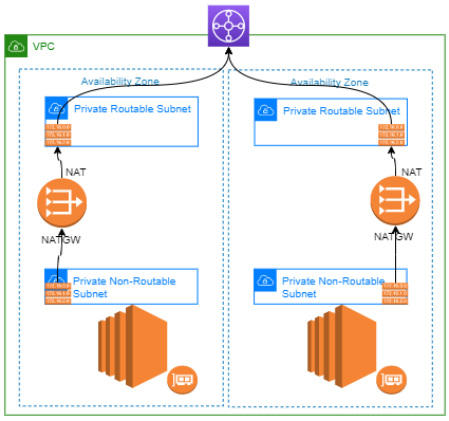


Figure 5: EKS Access using NATGW

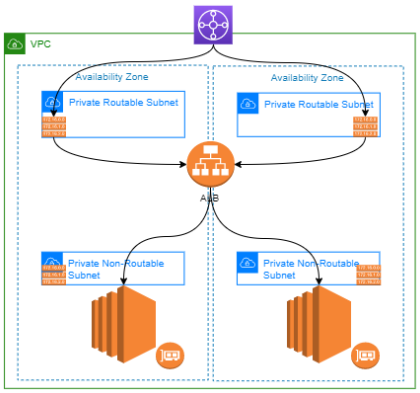


Figure 6: EKS Access using ELB

Ingress: For access from resources outside the VPC to the applications hosted in the EKS cluster an Application Load Balancer is installed with an ENI on a routable subnet in each availability zone. This ELB can use any of the EKS resources as the “target” in the ELB definition. When the ELB is in place and the targets are healthy, the EKS resources (targets) are reachable over the Aegon network where segmentation policies can be applied to the ELB to inspect/restrict access as necessary.

Consideration should still be given to the accessibility of other resources in the VPC to the EKS environment using the same controls document in the previous section:

Non-Routable Nodes and Containers Isolated to the VPC.

Routable Nodes with non-routable containers

In this model, the EKS Worker Nodes are installed on private, routable subnets and are reachable to/from the Aegon network if the proper segmentation policies are implemented. The Containers, however, are installed on private, non-routable subnets as in the previous two examples. The node has two ENIs in this model, with one ENI associated with each of the two subnets in the AZ. The container resources are isolated from the VPC where the cluster is deployed. Other than AWS resources reachable over VPC Endpoints, these resources cannot reach any resources outside the VPC **directly**. This model, like the previous model, offers the ability to allow communication to/from other Aegon networks for the isolated containers. In this model we deploy two separate network resources to facilitate communication for the containers. Communication to/from the containers is the same as documented in the previous section: Non-Routable Nodes and Containers with controlled access outside the VPC.

Node Egress: For node (EC2) access to resources outside the VPC, native routing in the VPC is used. Since the node is on a routable subnet, the route table of the subnet knows about the rest of the Aegon network and Internet. Therefore, the node can route traffic without the need for an intermediary device, like a NAT Gateway. Normal network segmentation policies apply to traffic leaving the VPC.

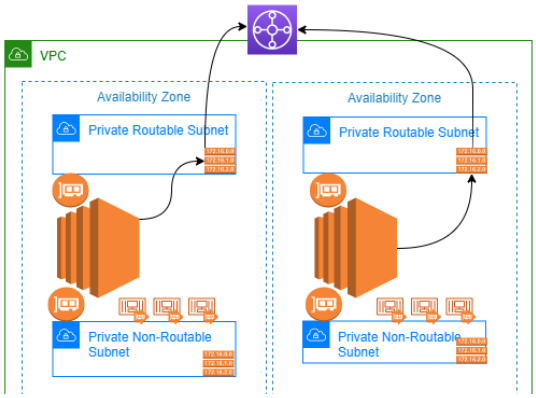


Figure 7: EKS Node access to resources outside the VPC

Node Ingress: For access from resources outside the VPC to the node (EC2), native routing in the VPC is used. Since the node is on a routable subnet, the Aegon network and Internet proxies know about the VPC network and can reach it without the need for an intermediary device, like an Application Load Balancer. Normal network segmentation policies apply to traffic entering the VPC.

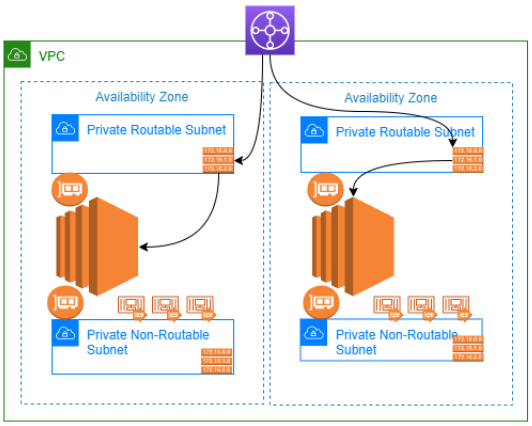


Figure 8: EKS Node access from resources outside the VPC

Consideration should still be given to the accessibility of other resources in the VPC to the EKS environment using the same controls document in the earlier section:

Non-Routable Nodes and Containers Isolated to the VPC.

The default behavior for the containers to NAT traffic through the primary interface of the node should be disabled. If this behavior is NOT disabled, and exception is required, and the environment needs to be treated like Routable Nodes and Containers.

Routable Nodes and Containers

In this model the nodes and containers are all deployed on private yet routable subnets and are reachable to/from the Aegon network if the proper segmentation policies are implemented. There is no need for intermediary devices (NATGW or ALB) to broker the communication. This model has the largest demand for routable IP Address as the container subnet typically needs to be large to account for autoscaling. While it is possible to implement this model with a single ENI on the node, a second ENI/subnet is recommended to allow the containers to scale into a subnet separate from the nodes primary ENI. This will allow the container subnet to be resized over time, without forcing the complete rebuild of the node.

Egress: For EKS node/container access to resources outside the VPC, native routing in the VPC is used. Since all the EKS resources are on routable subnets, the route tables of the subnets know about the rest of the Aegon network and Internet. Therefore, the EKS resources can route traffic without the need for an intermediary device, like a NAT Gateway. Normal network segmentation policies apply to traffic leaving the VPC.

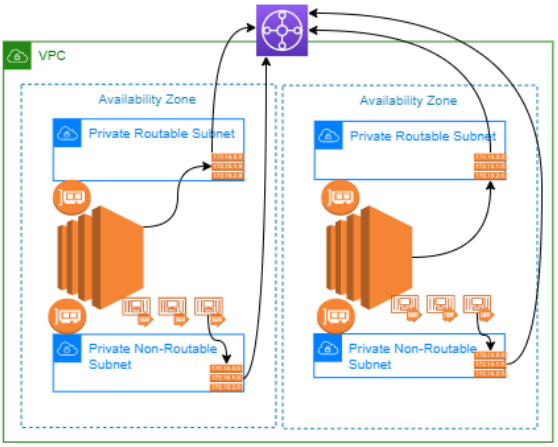


Figure 9 : EKS Traffic Egressing the VPC to Aegon

Ingress: For access from resources outside the VPC to the EKS node/container, native routing in the VPC is used. Since the EKS resources are on routable subnets, the Aegon network and Internet proxies know about the VPC network and can reach the resources without the need for an intermediary device, like an Application Load Balancer. Normal network segmentation policies apply to traffic entering the VPC

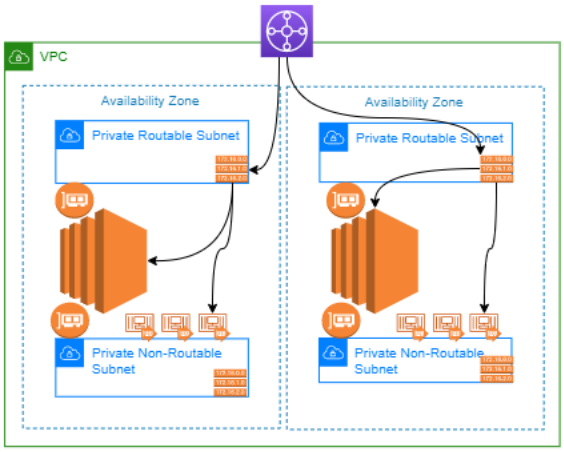


Figure 10 : Aegon traffic Ingress to the EKS VPC

Consideration should still be given to the accessibility of other resources in the VPC to the EKS environment using the same controls document in the earlier section:

Non-Routable Nodes and Containers Isolated to the VPC.

Note: An exception is required to use this pattern as it offers the least security control and has the largest impact on routable IP Address consumption.

Architectural Attributes

Interfaces

EKS provides several interfaces for monitoring and managing clusters, such as:

- **AWS Management Console**: EKS can be managed through the AWS Management Console, which provides a graphical user interface for cluster management, monitoring, and configuration.
- **AWS CLI**: The AWS Command Line Interface (CLI) allows users to interact with EKS clusters through command-line commands. It provides a set of commands for cluster creation, management, and monitoring.
- **AWS SDKs**: AWS Software Development Kits (SDKs) are available for various programming languages, such as Python, Java, and .NET. These SDKs provide libraries and APIs to interact with EKS clusters programmatically, enabling automation and integration with other systems.
- **Kubernetes Dashboard**: EKS integrates with the Kubernetes Dashboard, which is a web-based user interface for managing and monitoring Kubernetes clusters. It provides a visual representation of cluster resources, allows for resource management, and displays cluster metrics.
- **Third-Party Tools**: EKS is compatible with various third-party tools and utilities that specialize in monitoring and managing Kubernetes clusters. These tools can be integrated with EKS to provide advanced monitoring, logging, and troubleshooting capabilities. Helm can be used to deploy Istio, Prometheus, and the ELK stack onto a Kubernetes cluster.

Redundancy, Availability & Reliability

- **Redundancy**: EKS ensures redundancy by distributing the control plane across multiple availability zones (AZs). The control plane components, such as the API server, scheduler, etcd, are replicated and deployed in multiple AZs to provide high availability and fault tolerance. This redundancy helps to minimize the impact of failures and ensures that the cluster remains operational even if one AZ experiences issues.
- **Availability**: EKS offers high availability by leveraging the underlying infrastructure of AWS. The worker nodes in an EKS cluster are deployed across multiple AZs, allowing for workload distribution and fault tolerance. If one AZ becomes unavailable, the workloads can continue running in the remaining AZs without disruption. EKS also automatically detects and replaces unhealthy worker nodes, ensuring that the cluster remains available.
- **Reliability**: EKS is designed to be reliable by providing automated updates and patching for the control plane. AWS manages the underlying infrastructure, including the control plane components, ensuring that they are up to date with the latest security patches and updates. This helps to maintain the reliability and security of the cluster without requiring manual intervention.

Additionally, EKS integrates with other AWS services, such as Elastic Load Balancing, Amazon RDS, and Amazon EBS, which further enhance the reliability and availability of the cluster by providing scalable and resilient infrastructure components.

Scale, Elasticity & Capacity

EKS manages scale, elasticity, and capacity through its integration with AWS services and its ability to automatically adjust resources based on workload demands. Here are some key aspects:

- **Auto Scaling**: EKS leverages AWS Auto Scaling to automatically adjust the number of worker nodes in the cluster based on the workload. Auto Scaling allows the cluster to scale up or down to meet the demand, ensuring that there are enough resources available to handle the workload efficiently.
- **Elastic Load Balancing**: EKS integrates with AWS Elastic Load Balancing (ELB) to distribute incoming traffic across multiple worker nodes in the cluster. ELB automatically scales its capacity to handle the incoming traffic, ensuring that the cluster can handle high traffic loads without any disruption.
- **Integration with AWS Services**: EKS seamlessly integrates with other AWS services, such as Amazon EC2, Amazon EBS, and Amazon S3, to provide scalable and elastic storage solutions. This integration allows EKS to leverage the scalability and elasticity of these services to meet the storage requirements of the cluster.
- **Cluster Autoscaler**: EKS provides a Cluster Autoscaler feature that automatically adjusts the size of the cluster based on the resource utilization. It scales up the cluster when there is a shortage of resources and scales it down when the resources are underutilized. This ensures optimal resource allocation and efficient utilization of the cluster.
- **Capacity Planning**: EKS allows users to plan and manage the capacity of their clusters by providing insights into resource utilization and performance metrics. Users can monitor the cluster's resource usage and make informed decisions about scaling up or down based on the workload demands.

Overall, EKS manages scale, elasticity, and capacity by leveraging AWS services, integrating with auto scaling mechanisms, and providing tools for capacity planning and resource monitoring. This ensures that the cluster can efficiently handle varying workloads and adapt to changing resource requirements.

Even though EKS is highly available and scalable, there are certain constraints to be aware of.

- **Scalability limitations**: While EKS provides auto scaling capabilities for worker nodes, there may be limitations on the maximum number of nodes that can be scaled up in a cluster. This can impact on the ability to handle extremely large workloads or sudden spikes in demand. Additionally, if your scaling is based on Spot Instances, the availability of these instances may not be sufficient to handle your load when there is high demand for the resource.
- **Network configuration**: EKS requires proper network configuration to ensure connectivity between the cluster and other AWS services. This includes setting up VPCs, subnets, security groups, and configuring network policies. Incorrect network configuration can lead to connectivity issues and impact the performance and availability of the cluster. The worker nodes and pods need to have adequate network space to support the scaling needs of the application. This standard will discuss options for very large deployments.

Cost, Business & Support Models

The key elements that make up the cost associated with EKS include:

- **EC2 Instances**: EKS runs on EC2 instances (Worker Nodes), and the cost is based on the type and size of the instances used. Different instance types have different pricing structures. Spot instances can be used to provide a lower cost model, but the application needs to be structured to handle the capacity limits in times of high demand. Reserved instances are another option when the business understands the sustained compute demands for the application.
- **EBS Volumes**: EKS uses Elastic Block Store (EBS) volumes for persistent storage. The cost is based on the size and performance characteristics of the volumes.
- **Data Transfer**: EKS incurs costs for data transfer between the cluster and other AWS services or external networks. The cost depends on the amount of data transferred and where the data is transferred.
- **AWS Services Integration**: EKS integrates with various AWS services, such as Amazon RDS, Amazon S3, and AWS CloudWatch. The cost associated with using these services will be included in the overall cost of EKS.
- **Load Balancers**: EKS utilizes AWS Elastic Load Balancing (ELB) to distribute incoming traffic. The cost of ELB is based on the number of load balancers and the amount of data processed.
- **NAT Gateways**: EKS utilizes NAT Gateways to connect to the Aegon network outside the VPC. There is persistent cost associated with each NAT Gateway along with data transfer and processing cost. A separate NAT Gateway is required for each AZ.
- **Support and Management**: AWS offers different support plans for EKS, ranging from basic to enterprise-level support. The cost of support will depend on the chosen plan.

If AWS Fargate is used for EKS, the cost considerations change.

User Access & Roles

IAM (Identity and Access Management) is used to provide and secure access to EKS (Elastic Kubernetes Service) in the following ways:

- **User and Group Management**: IAM allows you to create and manage users and groups. You can assign IAM users or groups specific permissions to access and manage EKS resources.
- **Fine-Grained Permissions**: IAM policies can be created to grant or restrict access to specific EKS resources, such as clusters, nodes, or namespaces. These policies can be attached to IAM users, groups, or roles.
- **Integration with Kubernetes RBAC**: IAM can be integrated with Kubernetes RBAC, allowing you to map IAM roles to Kubernetes roles and bind them to specific users or groups. This provides granular control over access to Kubernetes resources within the EKS cluster.
- **Audit and Logging**: IAM provides detailed logging and auditing capabilities, allowing you to track and monitor access to EKS resources. This helps in maintaining security and compliance.
- **Secure Authentication**: IAM provides secure authentication mechanisms, such as IAM users, IAM roles, or federated identities (e.g., through SAML or OpenID Connect). This ensures that only authorized entities can access the EKS cluster.
- **Role-Based Access Control (RBAC)**: IAM roles can be created and associated with EKS clusters. These roles define the permissions and policies that control access to EKS resources. IAM roles can be assigned to users, groups, or even other AWS services.

By leveraging IAM, you can ensure that only authorized users or services have access to your EKS cluster and its resources, thereby enhancing the security and control of your containerized applications.

Federated Roles

See **Secure Authentication** above.

Access Control Policies

See **RBAC** above.

Restrictions

Use of EKS is not restricted from a development perspective. However, there are strict guidelines on how EKS should be implemented at Aegon to meet our security standards as described in this document.

Permitted Regions

EKS is only support in AWS regions supported by GTS. Currently this is:

- EU-WEST-1
- US-EAST-1
- US-EAST-2

Use Cases

Describe potential use cases. These can be references to provider use cases (such as linking to AWS documentation), or to Aegon Reference Architectures and Architecture Patterns.

The common use cases for deploying EKS (Elastic Kubernetes Service) include:

- **Application deployment**: EKS allows you to deploy and manage containerized applications using Kubernetes. It provides a scalable and reliable platform for running your applications, ensuring high availability and fault tolerance.
- **Microservices architecture**: EKS is well-suited for deploying microservices-based architectures. It allows you to break down your application into smaller, independent services that can be deployed and scaled individually.
- **Continuous integration and deployment** (CI/CD): EKS integrates well with CI/CD pipelines, enabling you to automate the deployment and release of your applications.
- **Hybrid and multi-cloud deployments**: EKS supports hybrid and multi-cloud architectures, allowing you to run your Kubernetes clusters across multiple regions or even different cloud providers. This provides flexibility and avoids vendor lock-in.
- **Big data processing**: EKS can be used for running big data processing workloads, such as Apache Spark or Apache Hadoop. It provides the necessary scalability and resource management capabilities to handle large-scale data processing tasks.
- **Machine learning and AI**: EKS can be leveraged for running machine learning and AI workloads. You can use frameworks like TensorFlow or PyTorch to train and deploy models at scale, taking advantage of the distributed computing capabilities of Kubernetes.

Data & Governance

Define the data that is used by each component. This includes the data's format, its location, and its security requirements. The data should be defined in a way that is consistent with the overall architecture, and that allows data to be easily shared between components.

Data governance concerns with using EKS (Elastic Kubernetes Service). Some of the common concerns include:

- **Data Security**: As EKS involves deploying and managing containerized applications, it is important to ensure that sensitive data is properly secured within the containers. This includes implementing encryption mechanisms, access controls, and secure communication channels.
- **Data Privacy**: EKS may involve processing and storing data that is subject to privacy regulations, such as personally identifiable information (PII) or sensitive business data. It is crucial to ensure compliance with data privacy laws and regulations, such as GDPR or HIPAA, when using EKS.
- **Data Residency**: Organizations may have specific requirements regarding where their data is stored and processed. When using EKS, it is important to consider the data residency requirements and ensure that the data is stored and processed in compliant locations.
- **Data Loss Prevention**: EKS involves managing and scaling containerized applications, which can introduce complexities in data backup and recovery processes. It is important to have proper data backup and disaster recovery mechanisms in place to prevent data loss in case of failures or outages.
- **Data Access Controls**: EKS integrates with IAM (Identity and Access Management) for access control. It is important to properly configure IAM roles, permissions, and policies to ensure that only authorized users or services have access to the data within the EKS cluster.
- **Data Monitoring and Auditing**: EKS provides logging and auditing capabilities through IAM. It is important to monitor and audit access to EKS resources to detect any unauthorized access or data breaches. This helps in maintaining data governance and compliance.

Data Classification

Refer to the ITCF Framework and ICTF P04.01.02 Information Security Classification Standard for guidance on data classification the controls for processing, storing and transmitting data.

- ***Public**: Can be used to transfer public files, such as marketing materials or product documentation.*
- ***Internal**: Can be used to transfer internal files, such as personal information in a defined context or operational guidelines.*
- ***Confidential**: Can be used to transfer sensitive files, such as personal information related to employees, business/marketing plans.*
- ***Strictly Confidential**: Can be used to transfer very sensitive files, such as healthcare data or government classified information, trade secrets, strategic plans.*
    - ***PII**: Can be used to transfer personally identifiable information (PII), such as names, addresses, and Social Security numbers.*
    - ***HIPAA:** Is HIPAA eligible, meaning it can be used to transfer healthcare data in compliance with the Health Insurance Portability and Accountability Act (HIPAA).*

EKS in and of itself does not have company data. The applications built upon EKS will have individual data classifications based on the type/amount of data being used or exposed by the application. Given that the configuration and controls associated with governing EKS might be sensitive, we will consider the component to have an “Internal” classification.

Guardrails

List the guardrails that need to be implemented to properly control the use of the component.

No Public Access

The VPC Subnets where EKS is deployed should have no public access (IGW). By default, we do not allow IGW or public subnets in AWS VPCs, but EcoSystem accounts have developer roles with additional privileges. Runbooks should ensure the EKS infrastructure is only inflated in private subnets.

Container NAT to the Node Primary IP Address is Disabled

The default behavior for EKS containers is to NAT outbound traffic behind the primary IP Address of the EKS Node (EC2). In order to keep the containers isolated, this default setting should be disabled. This will force any outbound communications through a NAT Gateway, which can be properly inspected and segmented using network segmentation controls.

Observability

Through comprehensive logging and monitoring, observability provides deep insights into the applications and infrastructure. This visibility is essential for ensuring high availability, optimizing resource utilization, and improving security—all pivotal in maintaining business continuity and meeting service-level agreements (SLAs).

To achieve effective observability within an EKS environment, it's essential to monitor various metrics and logs:

Performance Metrics: Key indicators such as CPU, memory, disk usage, network throughput, and latency are critical for identifying resource bottlenecks and ensuring appropriate scaling.

Workload-specific Metrics: Monitoring deployed applications' request rates, error rates, and response times is vital for understanding the end-user experience.

Health Checks and Logs: Insights from system and application logs are invaluable for identifying performance-affecting events and conditions. Logs from pods, services, and ingress controllers are particularly valuable.

Cluster State Information: Monitoring changes in Kubernetes resources like deployments, nodes, and pods helps diagnose system issues and understand cluster dynamics.

Container Vulnerabilities: Container images should be scanned on a regular basis to ensure the images do not contain any vulnerabilities.

How Observability Can Be Performed

Observability in a mature K8 estates involves a blend of built-in tools and external solutions that collectively gather, aggregate, and analyze data:

Helm: Helm plays a crucial role in deploying and managing Kubernetes applications, including those for observability. By using Helm, we can ensure that tools such as monitoring solutions and service meshes are deployed consistently across all environments, which is vital for maintaining reliable monitoring and logging practices.

Istio: As a service mesh, Istio provides automatic telemetry data that includes detailed metrics, logs, and traces of all communications between services. This built-in functionality enhances observability without requiring changes to the microservices themselves, making it an essential component in understanding service behavior and diagnosing issues within the mesh.

Prometheus: A tool that can complement existing logging solutions by focusing on metrics aggregation is crucial for a unified performance and health view. It provides real-time monitoring and alerting based on metrics from various sources within the cluster, aiding in performance tuning and predictive scaling decisions.

Integrated EKS Tools: AWS services like CloudWatch and CloudTrail can provide fundamental monitoring and auditing capabilities.

Container Insights: For more granular observability, Container Insights collects, aggregates, and summarizes metrics and logs from EKS clusters and their applications.

Centralized Logging: With existing solutions like Splunk for security logging and ELK/Neptune for application logging, a centralized logging approach helps aggregate logs from various cluster sources, simplifying analysis and troubleshooting.

How Observability Tools Work Together

Tool	Role in Observability	Interaction with Other Tools
Helm[1]	Manages the deployment of Kubernetes applications including observability tools like Prometheus, Istio, ELK, and others.	Deploys and manages Prometheus, Istio, ELK stacks. Ensures consistent configurations across deployments.
Istio[1]	Provides automatic telemetry data (metrics, logs, traces) for microservices in the service mesh.	Feeds detailed service-level metrics and logs into Prometheus and ELK for analysis. Uses Helm for deployment.
Splunk	Platform for searching, analyzing, and visualizing the machine-generated data gathered from the IT infrastructure.	Can ingest metrics and logs from Prometheus and Istio for advanced querying and real-time alerting.
ELK/Neptune	Elasticsearch for storing and indexing logs, Logstash for processing, and Kibana for visualizing data.	Can receive logs directly from Istio and other Kubernetes components. Prometheus data can be visualized in Kibana.
Prometheus[1]	Monitors and collects metrics from configured targets at specified intervals, offering a powerful querying language for analysis.	Metrics data from Prometheus can be used in Splunk and visualized in Kibana. Istio can provide service mesh metrics directly to Prometheus.

Deployment and Configuration: Helm can be used to deploy Istio, Prometheus, and the ELK stack onto a Kubernetes cluster. Helm ensures that these tools are configured correctly and consistently, simplifying the management of different observability components within the cluster.

Data Collection and Aggregation: Istio collects detailed metrics and logs at the service level within its service mesh. These logs and metrics are then pushed to Prometheus and the ELK stack. Prometheus collects metrics from Istio as well as other Kubernetes resources, providing a comprehensive view of the infrastructure's health and performance.

Data Analysis and Visualization: Splunk and ELK provide powerful platforms for data analysis and visualization. Splunk can ingest data from Prometheus for complex queries and real-time analysis. Meanwhile, ELK's Kibana can be used to create visualizations and dashboards from data collected by Prometheus and Istio, offering insights into application and infrastructure health.

Alerting and Monitoring: Prometheus includes an alerting module that can trigger alerts based on the metrics it gathers. These alerts can be sent to directly or integrated into other systems for automated incident response. Splunk can also configure alerts based on the log data it analyzes, providing another layer of monitoring.

1 – These products may not be approved at Aegon as of this writing, but are common products used by mature K8 hosting solutions.

Product / Component Selection

When to use, or not use this component. Include references to alternative products if possible.

Other cloud providers offer EKS alternatives with similar capability (i.e. Azure AKS, Google GKE). In addition, there are other products that support Kubernetes deployment (i.e. Docker, OpenShift)

Feature Comparison

Preferably a table of features in the same component-class, identifying key features, advantages and disadvantages based on the Architectural Attributes above.

Feature	EKS	OpenShift	Docker
Managed Kubernetes	Supported	Supported	
Automatic Scaling	Supported	Supported	Supported
Cloud Integration	Supported	Supported	Supported
High Availability	Supported	Supported	Supported
Monitoring	Supported	Supported	Supported
Logging	Supported	Supported	Supported
Community Support	Better than most	Better than Docker	
Integration with AWS	Native	Supported with own integration tools	Supported
Flexibility and Control	Good	Feature rich	Better container control

One thing to note, EKS is specific to AWS and vendor lock-in is if our direction moves away from AWS.

Exceptions

EKS is not suitable for deployment in Azure or Google. These cloud providers have their own versions of Kubernetes management platforms.

Usage in Aegon

Related Component Standards

Such as other products in the same category that have Component Standards

1. Component Standard name and hyperlink
2. Component Standard name and hyperlink

Architectural Patterns

A list of links to Architectural Patterns based on this component.

1. Architectural Pattern name and hyperlink
2. Architectural Pattern name and hyperlink

Reference Architecture

A list of links to Reference Architectures that consume this component.

1. Reference Architecture name and hyperlink
2. Reference Architecture name and hyperlink

Solution Architectures

A list of links to Solution Architectures that consume this component.

1. Solution Architecture name and hyperlink
2. Solution Architecture name and hyperlink